

编码规范

技术规范文件

保险公司 技术开发部
内部文档 · 仅供授权人员查阅

目录

- 1. 概述
 - 1.1 目的
 - 1.2 适用范围
- 2. Java 编码规范
 - 2.1 命名规范
 - 2.2 代码格式
 - 2.3 编程规约
 - 2.4 最佳实践
- 3. Python 编码规范
 - 3.1 命名规范
 - 3.2 代码格式
 - 3.3 编程规约
 - 3.4 最佳实践
- 4. Go 编码规范
 - 4.1 命名规范
 - 4.2 编程规约
 - 4.3 最佳实践
- 5. Spring Boot 规范
 - 5.1 项目结构
 - 5.2 分层规约
 - 5.3 配置管理
- 6. React 规范
 - 6.1 项目结构
 - 6.2 组件规范
 - 6.3 代码示例
- 7. 数据库对象命名规范
 - 7.1 命名规则
 - 7.2 表名规范
 - 7.3 字段规范
 - 7.4 SQL 规范

编码规范

> 参考：阿里巴巴 Java 开发手册、Google Style Guides、腾讯编码规范、Uber Go Style Guide

1. 概述

1.1 目的

统一技术开发部各语言、各框架的编码风格，提高代码可读性、可维护性和可审查性，降低团队协作成本。

1.2 适用范围

本规范覆盖 Java、Python、Go 三种主要编程语言，Spring Boot / React 框架，以及数据库对象命名。

2. Java 编码规范

2.1 命名规范

元素	规范	示例
类/接口	UpperCamelCase	PolicyServiceImpl
方法	lowerCamelCase	getPolicyByNo()
常量	UPPER_SNAKE_CASE	MAX_RETRY_COUNT
变量	lowerCamelCase	policyNo
包名	全小写，分隔	com.company.insurance.underwriting
抽象类	以 Abstract 开头	AbstractPolicyValidator
异常类	以 Exception 结尾	PolicyNotFoundException
测试类	以 Test 结尾	PolicyServiceTest

2.2 代码格式

- 缩进：4 空格（禁止 Tab）
- 行宽：不超过 120 字符
- 大括号：K&R 风格（左大括号不换行）
- 空行：类成员间用空行分隔，方法间 1 空行

2.3 编程规约

异常处理：

```
// 异常处理
if (policy == null) {
    throw new PolicyNotFoundException("异常: " + policyNo);
}
// 异常处理
try {
    // ...
} catch (Exception e) {
    // 异常处理
    log.error("异常", e);
}
```

集合处理：

```
// 集合处理
List<String> policyNoList = new ArrayList<>();
```

```
// 使用 Stream API 处理
List<String> activePolicies = policyList.stream()
    .filter(p -> "ACTIVE".equals(p.getStatus()))
    .map(Policy::getPolicyNo)
    .collect(Collectors.toList());
```

并发处理：

- 线程池不允许使用 `Executors.newFixedThreadPool()`，必须通过 `ThreadPoolExecutor` 明确参数
- 使用 `CompletableFuture` 而非手动 `FutureTask`
- 涉及账务操作必须加分布式锁（Redis/etcd），不可依赖本地锁

2.4 最佳实践

- 使用 Lombok 减少样板代码
- 使用 MapStruct 处理对象转换
- 使用 Builder 模式构造复杂对象
- 日志必须使用 SLF4J 门面 + Logback 实现

3. Python 编码规范

3.1 命名规范

元素	规范	示例
模块名	lowercase_with_underscores	policy_service.py
类名	UpperCamelCase	PolicyService
函数/方法	lowercase_with_underscores	get_policy_by_no()
变量	lowercase_with_underscores	policy_no
常量	UPPER_SNAKE_CASE	MAX_RETRY_COUNT
私有方法	前缀单下划线	_validate_policy()

3.2 代码格式

- 缩进：4 空格
- 行宽：不超过 88 字符（Black 默认值）
- 必须使用 Black + isort 做代码格式化
- 必须使用类型注解

3.3 编程规约

```
from typing import Optional
from decimal import Decimal
class PolicyService:
    """PolicyService"""
    def get_policy(self, policy_no: str) -> Optional[Policy]:
        """根据 policy_no 获取 Policy"""
        pass
    def calculate_premium(
        self,
        base_amount: Decimal,
        loading_rate: Decimal,
    ) -> Decimal:
        """计算保费"""
        # 返回 Decimal 类型
        return base_amount * (Decimal("1") + loading_rate)
```

3.4 最佳实践

- 使用 Pydantic 做数据校验和序列化
- 使用 SQLAlchemy 2.0 及以上版本（声明式映射）
- 使用 `match...case` 替代过多 `if/elif` (Python 3.10+)
- 异步代码优先使用 `asyncio + async/await`

4. Go 编码规范

4.1 命名规范

元素	规范	示例
包名	全小写单数形式	<code>policy</code> 、 <code>underwriting</code>
导出类型	PascalCase	<code>PolicyService</code>
非导出类型	camelCase	<code>policyRepo</code>
接口名	以 <code>er</code> 结尾	<code>PolicyRepository</code>
错误变量	以 <code>err</code> / <code>Err</code> 开头	<code>ErrPolicyNotFound</code>

4.2 编程规约

```
package policy
import "errors"
var ErrPolicyNotFound = errors.New("policy not found")
// PolicyService handles policy business logic
type PolicyService struct {
    repo PolicyRepository
}
// GetPolicy retrieves a policy by its number
func (s *PolicyService) GetPolicy(ctx context.Context, policyNo string) (*Policy, error) {
    if policyNo == "" {
        return nil, errors.New("policy number must not be empty")
    }
    return s.repo.FindByNo(ctx, policyNo)
}
```

4.3 最佳实践

- 错误处理：始终检查 `error` 返回值，不使用 `_` 忽略
- 使用 `errors.Is()` / `errors.As()` 做错误判断
- 使用 `sync/atomic` 而非 `sync.Mutex` 处理简单计数器
- 使用 `Wire` / `fx` 做依赖注入

5. Spring Boot 规范

5.1 项目结构

```
src/main/java/com/company/module/
├── controller/ # REST 接口
├── service/ # 业务逻辑
├── repository/ # 数据访问
├── model/entity/ # 实体类
├── model/dto/ # 数据传输对象
├── model/vo/ # 视图对象
├── config/ # 配置类
├── handler/ # 处理器
├── interceptor/ # 拦截器
```

```
└─ util/ # 工具
```

5.2 分层规约

层次	职责	禁止事项
Controller	参数校验、路由分发	不允许包含业务逻辑
Service	业务逻辑编排、事务管理	不允许直接操作数据库
Repository	数据访问	不允许跨表业务逻辑

- 事务注解 @Transactional 必须标注在 Service 层方法上
- 禁止在 Controller 层直接返回 Entity，必须转 DTO
- 参数校验使用 @Valid / @Validated

5.3 配置管理

- 环境配置：application-{profile}.yaml (dev/test/staging/prod)
- 密钥类配置使用配置中心（Nacos/Apollo）
- 禁止在代码中硬编码配置项

6. React 规范

6.1 项目结构

```
src/
├─ components/ # 组件
├─ pages/ # 页面
├─ hooks/ # Hooks
├─ services/ # API 接口
├─ stores/ # 状态
├─ types/ # TypeScript 类型
├─ utils/ # 工具
└─ constants/ # 常量
```

6.2 组件规范

- 使用 Function Component + Hooks，不使用 Class Component
- 使用 TypeScript，禁止使用 any 类型
- 组件单一职责，不超过 200 行
- 使用 React.memo 优化频繁渲染组件
- 状态管理：优先使用 React Context / Zustand，按需引入 Redux Toolkit

6.3 代码示例

```
import React, { useState, useCallback } from 'react';
import { Policy } from '@types/policy';
interface PolicySearchProps {
  onSearch: (keyword: string) => Promise<Policy[]>;
}
export const PolicySearch: React.FC<PolicySearchProps> = ({ onSearch }) => {
  const [keyword, setKeyword] = useState('');
  const [loading, setLoading] = useState(false);
  const handleSearch = useCallback(async () => {
    setLoading(true);
    try {
      await onSearch(keyword);
    } finally {
      setLoading(false);
    }
  }, [onSearch]);
```

```
}, [keyword, onSearch]));
return ( /* JSX */ );
};
```

7. 数据库对象命名规范

7.1 命名规则

对象	规范	示例
表	小写 + 下划线	t_policy_info
字段	小写 + 下划线	policy_no
主键	pk_{表名}	pk_t_policy_info
索引	idx_{表名}_{字段名}	idx_t_policy_info_policy_no
唯一约束	uk_{表名}_{字段名}	uk_t_policy_info_contract_no
外键	fk_{表名}_{表名}	fk_t_policy_t_product

7.2 表名规范

- 业务表前缀 t_，字典/配置表前缀 d_，日志表前缀 l_
- 使用名词单数形式：t_policy 而非 t_policies
- 关联表用 _rel 结尾：t_user_role_rel

7.3 字段规范

- 所有表必须包含 id（自增/Snowflake）、create_time、update_time、deleted（逻辑删除）
- 金额字段使用 DECIMAL(18,2)，不使用 FLOAT/DOUBLE
- 日期字段使用 DATETIME(3)（毫秒精度）
- 状态字段使用 TINYINT，注释标明枚举含义

7.4 SQL 规范

```
-- 查询所有已生效的政策
SELECT
p.policy_no,
p.premium
FROM
t_policy_info p
WHERE
p.status = 1
AND p.create_time >= '2024-01-01'
ORDER BY
p.create_time DESC;
```

- 禁止使用 SELECT *
- 复杂查询必须加 EXPLAIN 分析执行计划
- 涉及大表的 JOIN 必须评估索引使用情况

8. 附录

8.1 参考文档

- 《阿里巴巴 Java 开发手册》
- Google Java Style Guide
- Python PEP 8 / PEP 484

- Uber Go Style Guide
- 《Spring Boot 官方文档》
- React TypeScript Cheatsheet

8.2 工具配置

- Checkstyle / PMD (Java)
- Black + isort + flake8 + mypy (Python)
- golangci-lint (Go)
- ESLint + Prettier + TypeScript (React)
- SchemaSpy (数据库模型可视化)